

IEEE Standard for Floating-Point NumbersSem: 4th

Section: COA-CSE-G1 & COA-CSCE-G1

Faculty: Prof. Anil Kumar Swain

Lecture Summary

1. Introduction
2. IEEE Standard for Floating-Point Numbers
3. Arithmetic Operations on Floating-Point Numbers

1 Introduction

Virtually all modern computers follow the IEEE floating point standard in their representation of floating point numbers. The Java programming language types float and double use the IEEE single format and the IEEE double format respectively.

Floating point representation is based on exponential (or scientific) notation.

Floating Point Representation in Base 10 (decimal)

- In exponential notation, a nonzero real number x is expressed in decimal as

$$x = \pm M \times 10^E$$
- where $1 \leq M < 10$ and E is an integer. The numbers M and E are called the significant (commonly called mantissa) and the exponent respectively.
- Example: Positional representation of fractional number 34.211 is

10^3	10^2	10^1	10^0		10^{-1}	10^{-2}	10^{-3}	10^{-4}	Multiplier
		3	4	.	2	1	1		Number
3	2	1	0		-1	-2	-3	-4	Position

↑
Decimal Point

- The following are equivalent representations of **34.211**

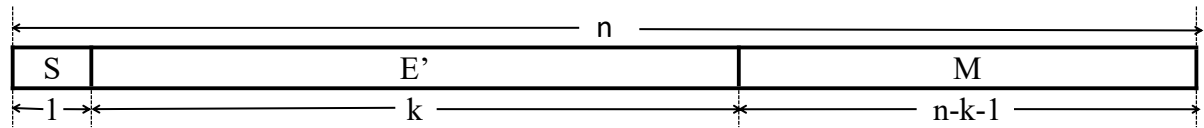
3	4	2	1	1	.	0				X	10 ⁻³
	3	4	2	1	.	1				X	10 ⁻²
		3	4	2	.	1	1			X	10 ⁻¹
			3	4	.	2	1	1		X	10⁰
				3	.	4	2	1	1		X 10 ¹
				0	.	3	4	2	1	1	X 10 ²
				0	.	0	3	4	2	1	1 X 10 ³

2 Floating Point Representation in Base 2 (Binary)

- The floating point numbers are stored in Mantissa (M) and Exponent (E) form.
- Most of the notations represent mantissa as normalized sign magnitude fraction.
- The normalized can be explicit or implicit.
- The exponent is denoted in biased form. The biased exponent is an unsigned number which can represent signed exponent of original number.

$$\text{True Exponent (E)} = \text{Biased Exponent (E')} - \text{Bias}$$

- The floating point number is stored in the following way.



S=Sign Bit=1 bit (0 for +, 1 for negative)

If k bits are reserved for Exponent, then Bias will be 2^{k-1} .

Biased Exponent will be $0 \leq E' \leq 2^k - 1$

M=Mantissa= Rest Bits=n-k-1

- In exponential notation, a nonzero real number x is expressed in binary as

$$x = \pm M \times 2^E$$

- The following are equivalent representations of **1011.01**

1 0 1 1 0 1 0 . 0	X 2^{-3}
1 0 1 1 0 1 . 0	X 2^{-2}
1 0 1 1 0 . 1	X 2^{-1}
1 0 1 1 . 0 1	X 2^0
1 0 1 . 1 0 1	X 2^1
1 0 . 1 1 0 1	X 2^2
1 . 0 1 1 0 1	X 2^3

- **Normalized floating-point numbers:** To efficiently use the bits available for the significand, it is shifted to the left until all leading 0's disappear (as they make no contribution to the precision). The value can be kept unchanged by adjusting the exponent accordingly. Moreover, as the MSB of the significand is always 1, it does not need to be shown explicitly. The significand could be further shifted to the left by 1 bit to gain one more bit for precision. The first bit 1 before the decimal point is implicit.

Normalized floating point numbers are expressed as

$$1.b_1b_2b_3...b_n \times 2^E, \text{ Where } E \text{ is the real exponent and } b_i \in \{0,1\}.$$

In the above example the last row that is 1.01101×2^3 is the normalized floating point number of the given floating point number 1011.01

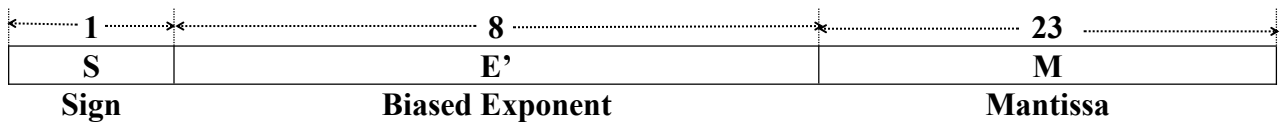
- **Reasons for normalization:**
 - ❖ Representation of each number unique.
 - ❖ No digits wasted on leading zeros.
 - ❖ Leading bit need not be stored (in binary system).
 - ❖ By normalization, highest precision can be achieved.

3 IEEE standard for representing floating-point numbers.

Format	Sign Bit	Exponent Bits	Mantissa Bits	Exponent Excess (Bias)
IEEE 32-bit (Single Precision)	1	8	23	127
IEEE 64-bit (Double Precision)	1	11	52	1023

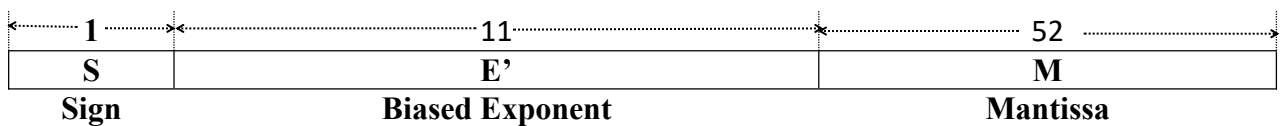
- The exponent field (E) is in plain binary format which also represents negative exponents with an encoding (like sign magnitude, 1's complement, 2's complement, etc.). The biased exponent (E') is found as follows:
- A bias of $(2^{n-1} - 1)$, where n is the number of bits used in real exponent (E), is added to the real exponent (E) to get biased exponent (E').

$$E' = E + \text{Bias}$$



IEEE 32-bit (Single Precision) Format

$$\text{Here } E' = E + 127$$



IEEE 64-bit (Double Precision) Format

$$\text{Here } E' = E + 1023$$

Examples

Converting to Floating Point

Q1. Express 36.5625_{10} as a 32-bit floating point number.

Solution

Step-1: Express original value in binary

$36.5625_{10} = 100100.1001_2$	<u>Rough Work</u>
--------------------------------	--------------------------

Step-2: Normalize the original value found in step-1.

$$100100.1001_2 = 1.001001001_2 \times 2^5$$

Step-3: Put the sign (here it is +) before the normalized value and determine S, E', and M.

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ +1.001001001_2 \times 2^5 \\ S & M & E \end{array}$$

As the number is +ve, so $S=0$

Mantissa = $M = 001001001$ (9 bit) so make it 23 bit, that is to add 14 zeros right to M. so

$$M = 001001001000000000000000_2$$

Original Exponent = $E = 5$

Biased Exponent = $E' = E + 127 = 5 + 127 = 132$

Find out the 8-bit binary equivalent of biases exponent = $E' = 132 = 10000100_2$

Step-4: Put S, E', and M together to form 32-bit binary result.

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 0 \ 10000100 \ 001001001000000000000000_2 \\ S & E' & M \end{array}$$

The above result in hexadecimal form is

$$01000010000100100100000000000000_2 = 0100 \ 0010 \ 0001 \ 0010 \ 0100 \ 0000 \ 0000 \ 0000_2$$

$$= 4 \quad 2 \quad 1 \quad 2 \quad 4 \quad 0 \quad 0 \quad 0_{16}$$

Q2. Represent $\frac{11}{2}$ in 32-bit IEEE standard format.

Solution

Step-1: Express original value in binary

$$\frac{11}{2} = 5.5 = 101.1$$

Step-2: Normalize the original value found in step-1.

$$101.1 = 1.011 \times 2^2$$

Step-3: Put the sign (here it is +) before the normalized value and determine S, E', and M.

$$+1.011 \times 2^2$$

Here S=0

$$M=011 \Rightarrow M=011\text{REST 20 ZEROS}$$

$$E=2 \Rightarrow E'=E+127=2+127=129 = 1000\ 0001$$

Step-4: Put S, E', and M together to form 32-bit binary result.

S	E'	M
0	1000 0001	011REST 20 ZEROS

Converting from Floating Point

Q1. What decimal value is represented by the following 32-bit floating point number?

11000001011110110000000000000000

Solution

Step-1: Locate S, E' and M in the given 32 bit binary number. If the given number is in hexadecimal first convert into binary and then locate S, E' and M. Express E' in decimal and find the real exponent E

From left to right

First bit is the sign bit (S), in this case S=1 so the number is -ve

Next 8 bit as biased exponent (E'), in this case $E' = 10000010 = 130$ (in decimal)

$$\Rightarrow E=E'-127=130-127=3$$

Rest 23 bit as mantissa (M), in this case $M = 11110110000000000000000$

Step-2: Put S,M and E together to form binary result (Don't forget the implied "1." on the left of the mantissa.)

$$\Rightarrow (+/-)1.M \times 2^E \Rightarrow -1.1111011 \times 2^3$$

Step-3: Express result in decimal.

$$-1.1111011_2 \times 2^3 = -1111.1011_2 = -15.6875_{10}$$